Experiment 'p_14_7_bubble_sort_seq' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_14_7_bubble_sort_seq
Natural language statement: Write a method that implements the bubble sort algorithm to sort an array of integers.
Method signature: p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)

Ensures

- `forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]`
- `multiset(inputs) == multiset(sorted)`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that implements the bubble sort algorithm to sort an array of integers.

The signature should be:

p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset
    ↪ (inputs) == multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnKoDo7xdT7cHYVTLlcJYwUdvkGft
Model: gpt-5.5-2026-04-23
Created at: 1780650433
Finish reason: stop
Usage: 322 tokens in, and 8689 tokens out

```
{
  sorted := inputs;
  var n := |sorted|;

  while n > 0
    invariant 0 <= n <= |sorted|
    invariant multiset(sorted) == multiset(inputs)
    invariant forall a, b :: n <= a < b < |sorted| ==> sorted[a] <= sorted[b]
    invariant forall a, b :: 0 <= a < n <= b < |sorted| ==> sorted[a] <= sorted[b]
    decreases n
  {
    var j := 0;

    while j + 1 < n
      invariant 0 <= j < n <= |sorted|
      invariant multiset(sorted) == multiset(inputs)
      invariant forall a, b :: n <= a < b < |sorted| ==> sorted[a] <= sorted[b]
      invariant forall a, b :: 0 <= a < n <= b < |sorted| ==> sorted[a] <= sorted[b]
      invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j]
      decreases n - j
```

```
{
  var before := sorted;
  var doSwap := before[j] > before[j + 1];

  if doSwap {
    assert 0 <= j < j + 1 < |before|;
    assert before == before[..j] + [before[j], before[j + 1]] + before[j + 2..];
    sorted := before[..j] + [before[j + 1], before[j]] + before[j + 2..];
    assert |sorted| == |before|;
    assert sorted[j] == before[j + 1];
    assert sorted[j + 1] == before[j];
    assert forall k :: 0 <= k < |sorted| && k != j && k != j + 1 ==> sorted[k] ==
        ↪ before[k];
    assert multiset(sorted) == multiset(before[..j]) + multiset([before[j + 1], before
        ↪ [j]]) + multiset(before[j + 2..]);
    assert multiset(before) == multiset(before[..j]) + multiset([before[j], before[j +
        ↪  1]]) + multiset(before[j + 2..]);
    assert multiset([before[j + 1], before[j]]) == multiset([before[j], before[j +
        ↪ 1]]);
    assert multiset(sorted) == multiset(before);
  } else {
    assert sorted == before;
  }

  assert |sorted| == |before|;
  assert multiset(sorted) == multiset(inputs);

  forall a, b | n <= a < b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    assert j + 1 < n <= a;
    if doSwap {
      assert sorted[a] == before[a];
      assert sorted[b] == before[b];
    } else {
      assert sorted == before;
    }
    assert before[a] <= before[b];
  }

  forall a, b | 0 <= a < n <= b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    assert j + 1 < n <= b;
    if doSwap {
      assert sorted[b] == before[b];
      if a == j {
        assert sorted[a] == before[j + 1];
        assert 0 <= j + 1 < n <= b < |before|;
        assert before[j + 1] <= before[b];
      } else if a == j + 1 {
```

```
        assert sorted[a] == before[j];
        assert 0 <= j < n <= b < |before|;
        assert before[j] <= before[b];
      } else {
        assert sorted[a] == before[a];
        assert 0 <= a < n <= b < |before|;
        assert before[a] <= before[b];
      }
    } else {
      assert sorted == before;
      assert before[a] <= before[b];
    }
  }

  forall k | 0 <= k < j + 1
    ensures sorted[k] <= sorted[j + 1]
  {
    if doSwap {
      assert sorted[j + 1] == before[j];
      if k == j {
        assert sorted[k] == before[j + 1];
        assert before[j + 1] <= before[j];
      } else {
        assert k < j;
        assert before[k] <= before[j];
        assert sorted[k] == before[k];
      }
    } else {
      assert sorted == before;
      assert before[j] <= before[j + 1];
      if k == j {
        assert before[k] <= before[j + 1];
      } else {
        assert k < j;
        assert before[k] <= before[j];
        assert before[k] <= before[j + 1];
      }
    }
  }

  j := j + 1;
}

var oldN := n;
assert j + 1 >= oldN;
assert j == oldN - 1;
var newN := oldN - 1;
assert 0 <= newN;

forall a, b | newN <= a < b < |sorted|
  ensures sorted[a] <= sorted[b]
```

```
    {
      if a == newN {
        assert b >= oldN;
        assert 0 <= a < oldN <= b < |sorted|;
        assert sorted[a] <= sorted[b];
      } else {
        assert oldN <= a < b < |sorted|;
        assert sorted[a] <= sorted[b];
      }
    }

    forall a, b | 0 <= a < newN <= b < |sorted|
      ensures sorted[a] <= sorted[b]
    {
      if b == newN {
        assert a < j;
        assert sorted[a] <= sorted[j];
        assert j == newN;
      } else {
        assert b >= oldN;
        assert 0 <= a < oldN <= b < |sorted|;
        assert sorted[a] <= sorted[b];
      }
    }

    n := newN;
  }
}
```

Final Program

```
method p_14_4_bubble_sort_seq(inputs: seq<int>) returns (sorted: seq<int>)
       ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
       ensures multiset(inputs) == multiset(sorted)
{
  sorted := inputs;
  var n := |sorted|;

  while n > 0
    invariant 0 <= n <= |sorted|
    invariant multiset(sorted) == multiset(inputs)
    invariant forall a, b :: n <= a < b < |sorted| ==> sorted[a] <= sorted[b]
    invariant forall a, b :: 0 <= a < n <= b < |sorted| ==> sorted[a] <= sorted[b]
    decreases n
  {
    var j := 0;

    while j + 1 < n
      invariant 0 <= j < n <= |sorted|
      invariant multiset(sorted) == multiset(inputs)
```

```
  invariant forall a, b :: n <= a < b < |sorted| ==> sorted[a] <= sorted[b]
  invariant forall a, b :: 0 <= a < n <= b < |sorted| ==> sorted[a] <= sorted[b]
  invariant forall k :: 0 <= k < j ==> sorted[k] <= sorted[j]
  decreases n - j
{
  var before := sorted;
  var doSwap := before[j] > before[j + 1];

  if doSwap {
    assert 0 <= j < j + 1 < |before|;
    assert before == before[..j] + [before[j], before[j + 1]] + before[j + 2..];
    sorted := before[..j] + [before[j + 1], before[j]] + before[j + 2..];
    assert |sorted| == |before|;
    assert sorted[j] == before[j + 1];
    assert sorted[j + 1] == before[j];
    assert forall k :: 0 <= k < |sorted| && k != j && k != j + 1 ==> sorted[k] ==
        ↪ before[k];
    assert multiset(sorted) == multiset(before[..j]) + multiset([before[j + 1], before
        ↪ [j]]) + multiset(before[j + 2..]);
    assert multiset(before) == multiset(before[..j]) + multiset([before[j], before[j +
        ↪  1]]) + multiset(before[j + 2..]);
    assert multiset([before[j + 1], before[j]]) == multiset([before[j], before[j +
        ↪ 1]]);
    assert multiset(sorted) == multiset(before);
  } else {
    assert sorted == before;
  }

  assert |sorted| == |before|;
  assert multiset(sorted) == multiset(inputs);

  forall a, b | n <= a < b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    assert j + 1 < n <= a;
    if doSwap {
      assert sorted[a] == before[a];
      assert sorted[b] == before[b];
    } else {
      assert sorted == before;
    }
    assert before[a] <= before[b];
  }

  forall a, b | 0 <= a < n <= b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    assert j + 1 < n <= b;
    if doSwap {
      assert sorted[b] == before[b];
      if a == j {
```

```
        assert sorted[a] == before[j + 1];
        assert 0 <= j + 1 < n <= b < |before|;
        assert before[j + 1] <= before[b];
      } else if a == j + 1 {
        assert sorted[a] == before[j];
        assert 0 <= j < n <= b < |before|;
        assert before[j] <= before[b];
      } else {
        assert sorted[a] == before[a];
        assert 0 <= a < n <= b < |before|;
        assert before[a] <= before[b];
      }
    } else {
      assert sorted == before;
      assert before[a] <= before[b];
    }
  }

  forall k | 0 <= k < j + 1
    ensures sorted[k] <= sorted[j + 1]
  {
    if doSwap {
      assert sorted[j + 1] == before[j];
      if k == j {
        assert sorted[k] == before[j + 1];
        assert before[j + 1] <= before[j];
      } else {
        assert k < j;
        assert before[k] <= before[j];
        assert sorted[k] == before[k];
      }
    } else {
      assert sorted == before;
      assert before[j] <= before[j + 1];
      if k == j {
        assert before[k] <= before[j + 1];
      } else {
        assert k < j;
        assert before[k] <= before[j];
        assert before[k] <= before[j + 1];
      }
    }
  }

  j := j + 1;
}

var oldN := n;
assert j + 1 >= oldN;
assert j == oldN - 1;
var newN := oldN - 1;
```

```
    assert 0 <= newN;

    forall a, b | newN <= a < b < |sorted|
      ensures sorted[a] <= sorted[b]
    {
      if a == newN {
        assert b >= oldN;
        assert 0 <= a < oldN <= b < |sorted|;
        assert sorted[a] <= sorted[b];
      } else {
        assert oldN <= a < b < |sorted|;
        assert sorted[a] <= sorted[b];
      }
    }

    forall a, b | 0 <= a < newN <= b < |sorted|
      ensures sorted[a] <= sorted[b]
    {
      if b == newN {
        assert a < j;
        assert sorted[a] <= sorted[j];
        assert j == newN;
      } else {
        assert b >= oldN;
        assert 0 <= a < oldN <= b < |sorted|;
        assert sorted[a] <= sorted[b];
      }
    }

    n := newN;
  }
}
```

Total Token Usage

Input tokens: 322
Output tokens: 8689
Reasoning tokens: 7168
Sum of 'total tokens': 9011

Experiment Timings

Overall Experiment started at 1780650432213, ended at 1780650632681, lasting 200468ms (200.47 seconds)
Iteration #1 started at 1780650432214, ended at 1780650632681, lasting 200467ms (200.47 seconds)